

Networks and Systems

Lectures 4 & 5 — Exercise 1: Sensor Network Packetisation
Network Performance Analysis | Electronic Systems, 8th Semester

Problem Statement

Note on notation. The original exercise statement uses the letter N for two distinct things: the number of sensors (which is a fixed given value of 8) and the number of data points per packet (which is the quantity to be determined). To avoid confusion, this document uses $n_s = 8$ for the number of sensors, and reserves N solely for the number of data points bundled into each packet.

A sensor network consists of $n_s = 8$ identical sensors sharing a single communication link. The setup is as follows:

- Each sensor generates **100 bits** of measurement data periodically, once every $T = 1$ ms.
- All sensors share a single communication line with a serialisation speed (link bandwidth) of $s = 1$ Mbps = 1000 kbps.
- Before transmission, each sensor **buffers** N consecutive data samples and then sends them as a single packet. Each packet consists of the N data samples (each 100 bits) plus a fixed overhead (header, error detection, etc.) of $O = 100$ bits.

Tasks:

1. Determine the **optimal value of N** — the number of data points per packet — taking into account both the end-to-end delay experienced by the data and the bandwidth utilisation of the shared link.
2. For an **arbitrary N** , derive the **DNC arrival constraint** (arrival curve) for a single sensor, and then the **aggregate arrival curve** for all $n_s = 8$ sensors combined.

Background: Why Does N Matter?

The choice of N controls a fundamental trade-off in packetised sensor networks.

Large N :

- Each packet carries more data, so the overhead $O = 100$ bits is spread over more payload bytes. The *overhead fraction* $O/L = 100/[100(N + 1)] = 1/(N + 1)$ shrinks, meaning the link is used more efficiently (less bandwidth wasted on headers).
- However, a sensor must wait longer before it has accumulated N samples. The most recently generated data point sits in the buffer for up to $(N - 1) \times T$ milliseconds

before the packet is sent. **Delay increases.**

Small N :

- Packets are sent more frequently and contain fewer samples, so individual data points are transmitted sooner. **Delay decreases.**
- But each packet carries a large overhead relative to its payload. More packets per second means more overhead bytes per second, consuming a larger share of the link capacity. If N is too small the total offered load exceeds the link bandwidth and the system becomes **unstable** (infinite queues).

Both the bandwidth and delay depend on N in opposite directions — the solution is to find the smallest N that keeps the system stable, since that also minimises delay.

Task 1 — Optimal Packet Size N

Step 1: Derive the packet length L

Each packet from one sensor contains N data samples of 100 bits each, plus $O = 100$ bits of overhead:

$$L = N \times 100 + O = 100N + 100 = 100(N + 1) \quad [\text{bits}].$$

- L — total packet length in bits.
- N — number of data samples bundled per packet (to be determined).
- 100 bits — size of one data sample.
- $O = 100$ bits — fixed overhead (header, error detection, etc.).

Step 2: Find the inter-packet period P

Because a sensor generates one sample every $T = 1$ ms and must collect N samples before sending, the inter-packet period is:

$$P = N \times T = N \times 1 \text{ ms} = N \text{ ms}.$$

- P — time between successive packet transmissions from one sensor.
- N — the buffering window (number of samples to collect).
- $T = 1$ ms — sampling period of each sensor.

Step 3: Bandwidth constraint

Each of the $n_s = 8$ sensors sends one packet of L bits every P ms. The total offered bit rate to the shared link is:

$$r_{\text{total}} = n_s \cdot \frac{L}{P} = 8 \cdot \frac{100(N + 1)}{N \text{ ms}} = \frac{800(N + 1)}{N} \text{ kbps}.$$

- r_{total} — total bit rate offered to the link by all sensors.

- $n_s = 8$ — number of sensors.
- $L = 100(N + 1)$ bits — packet size.
- $P = N$ ms — inter-packet period per sensor.

For the link not to become overloaded, the offered rate must not exceed the link speed. The utilisation ρ must satisfy:

$$\rho = \frac{r_{\text{total}}}{s} = \frac{800(N + 1)}{N \times 1000} = \frac{0.8(N + 1)}{N} \leq 1.$$

- ρ — link utilisation (dimensionless); must be at most 1 for the offered load not to exceed link capacity.
- $s = 1000$ kbps — link capacity.

Solving for N :

$$\frac{0.8(N + 1)}{N} \leq 1 \implies 0.8N + 0.8 \leq N \implies 0.8 \leq 0.2N \implies N \geq 4.$$

So the minimum integer satisfying the bandwidth constraint is $N = 4$, which gives exactly $\rho = 0.8 \times 5/4 = 1.0$ — the link is fully utilised. Under the idealised assumption of this exercise (perfectly periodic sensors, no jitter, no burst) this is mathematically valid: the offered rate equals the capacity and the link can just keep up.

$N = 4$ works in theory, but is not recommended in practice.

Designing to $\rho = 1.0$ means the link has **zero spare capacity**. In any realistic system this is problematic:

- Sensors are not perfectly synchronised — small timing jitter means packets from multiple sensors can collide at the link simultaneously, creating momentary bursts that exceed the link rate and causing queues to grow.
- Any other background traffic (management frames, retransmissions) instantly pushes utilisation above 1 and the queue grows without bound.
- Clock drift over time means the “perfectly periodic” assumption gradually breaks down.

For these reasons, a utilisation of $\rho = 1.0$ is not a robust operating point. The next valid value, $N = 5$ ($\rho = 0.96$), provides a small but meaningful safety margin and is the **recommended practical choice**.

Step 4: Worst-case end-to-end delay

The total delay a data sample experiences has three additive components:

(a) Packetisation delay d_{pkt} : A data sample generated at the very start of a collection window must wait for the remaining $N - 1$ samples to arrive before the packet can be dispatched:

$$d_{\text{pkt}} = (N - 1) \times T = (N - 1) \text{ ms.}$$

(b) Queuing delay d_{queue} : In the worst case, all other $n_s - 1 = 7$ sensors happen to finish their collection windows at the same time and also have a packet ready. Our packet must wait for each of those 7 packets to be transmitted before it can start:

$$d_{\text{queue}} = 7 \times \frac{L}{s} = 7 \times \frac{100(N + 1)}{1000} \text{ ms} = 0.7(N + 1) \text{ ms.}$$

(c) Serialisation delay d_{ser} : The time to push the packet itself onto the wire:

$$d_{\text{ser}} = \frac{L}{s} = \frac{100(N + 1)}{1000} \text{ ms} = 0.1(N + 1) \text{ ms.}$$

Total worst-case end-to-end delay:

$$D(N) = d_{\text{pkt}} + d_{\text{queue}} + d_{\text{ser}} = (N - 1) + 0.7(N + 1) + 0.1(N + 1) = (N - 1) + 0.8(N + 1).$$

Expanding:

$$D(N) = 1.8N - 0.2 \quad [\text{ms}].$$

- $D(N)$ — worst-case delay in milliseconds as a function of N .
- The coefficient 1.8 and constant -0.2 come from combining the three delay terms above.
- $D(N)$ is **strictly increasing** in N : every extra data point bundled into a packet adds 1.8 ms to the worst-case delay.

Step 5: Determine the optimal N

We now have two constraints pulling in opposite directions:

N	$\rho = 0.8(N + 1)/N$	Stable?	$D(N)$ [ms]
1	1.60	No	1.6
2	1.20	No	3.4
3	1.07	No	5.2
4	1.00	Yes — $\rho = 1$ (theoretical min.)	7.0
5	0.96	Yes — recommended	8.8
6	0.93	Yes	10.6
7	0.91	Yes	12.4
8	0.90	Yes	14.2

Theoretical minimum: $N = 4$ ($\rho = 1.0$, lowest delay satisfying the bandwidth constraint).

- Packet size: $L = 100 \times 5 = 500$ bits
- Inter-packet period per sensor: $P = 4$ ms
- Link utilisation: $\rho = 1.00$ (link fully loaded)
- Worst-case delay: $D(4) = 7.0$ ms

Practical recommendation: $N = 5$ — avoids the zero-headroom problem at $\rho = 1$ while adding only 1.8 ms of extra delay.

- Packet size: $L = 100 \times 6 = 600$ bits
- Inter-packet period per sensor: $P = 5$ ms
- Link utilisation: $\rho = 0.96$
- Worst-case delay: $D(5) = 8.8$ ms

The figure below shows utilisation and delay versus N . Both $N = 4$ (blue highlight, $\rho = 1.0$) and $N = 5$ (green highlight, recommended) are marked.

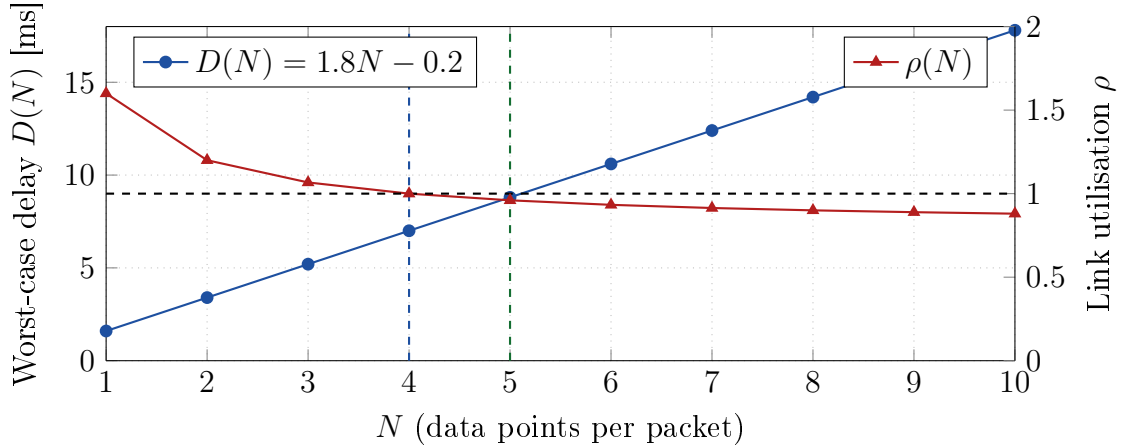


Figure 1: Delay $D(N)$ (blue curve, left axis) and utilisation $\rho(N)$ (red curve, right axis) as a function of N . The black dashed horizontal line marks $\rho = 1$. The **blue vertical dashed** line marks $N = 4$: the theoretical minimum satisfying $\rho \leq 1$ (link exactly fully loaded). The **green vertical dashed** line marks $N = 5$: the recommended practical choice with a small headroom ($\rho = 0.96$). For $N \leq 3$ the offered load exceeds the link capacity.

Task 2 — DNC Arrival Curves

Deterministic Network Calculus (DNC) characterises traffic flows using **arrival curves**. An arrival curve $\alpha(t)$ is an upper bound on the amount of data any flow can inject into

the network in any time window of length t :

$$A(s, s + t) \leq \alpha(t) \quad \forall s \geq 0, t \geq 0,$$

- $A(s, s + t)$ — number of bits arriving in the interval $(s, s + t]$.
- $\alpha(t)$ — the arrival curve: a non-decreasing function bounding the worst-case arrivals in any window of length t .

The most common arrival curve model is the **token bucket** (also called leaky bucket), written $\alpha(t) = r \cdot t + b$, where:

- r — the **token rate** (long-term average rate); tokens accumulate in a bucket at rate r and each bit of data consumes one token. This bounds how fast data can be sent in a sustained way.
- b — the **bucket size** (burst parameter); the maximum number of bits that can be sent instantaneously (i.e. the maximum burst in any arbitrarily short window).

Single-sensor arrival curve

Each sensor is a **periodic on-off source**: it is silent for $P = N$ ms while buffering samples, then injects one packet of $L = 100(N + 1)$ bits all at once, then is silent again for another P ms.

The tightest token-bucket arrival curve for this source is found by asking: *what is the maximum number of bits this sensor can send in any window of length t ?*

- **Burst b** : In any arbitrarily short window ($t \rightarrow 0$), the worst case is one packet arriving all at once:

$$b = L = 100(N + 1) \text{ bits.}$$

- **Rate r** : Over a long window the average sending rate is one packet per period:

$$r = \frac{L}{P} = \frac{100(N + 1)}{N \text{ ms}} = \frac{100(N + 1)}{N} \text{ kbps.}$$

The single-sensor arrival curve is therefore:

$$\alpha_s(t) = \frac{100(N + 1)}{N} \cdot t + 100(N + 1) \quad [\text{bits, with } t \text{ in ms}].$$

- $\alpha_s(t)$ — worst-case bits sent by one sensor in any interval of length t .
- $\frac{100(N+1)}{N}$ — token rate in kbps (bits per ms); the sustainable long-term rate of the sensor.
- $100(N + 1)$ bits — burst size; one full packet can arrive instantaneously.

Physical meaning of b and $r \cdot t$ in the token bucket.

The burst term b represents the worst-case instantaneous injection of bits — the amount of data that can arrive in an arbitrarily short window. For our sensor this is exactly one full packet L , because the sensor accumulates N samples silently and then sends them all at once. Concretely: imagine the link briefly goes down and then

recovers. While the link was down, the sensor kept generating samples and filling its buffer. The moment the link comes back, a fully-loaded packet (L bits) is released instantly. The bucket size $b = L$ bounds exactly this burst.

The rate term $r \cdot t$ represents the ongoing, sustained accumulation of data over time. After the initial burst, the sensor keeps generating one data point every $T = 1$ ms, so data accumulates at the long-term rate $r = L/P$. Over a window of length t at most $r \cdot t$ additional bits can be produced, regardless of when in that window they arrive.

Together, $\alpha_s(t) = r \cdot t + b$ says: *in any time window of length t , a single sensor sends at most b bits as an initial burst plus $r \cdot t$ bits from sustained generation.*

Aggregate arrival curve for all $n_s = 8$ sensors

When multiple flows share a link, DNC computes the aggregate arrival curve by **summing** the individual arrival curves. This is a valid (worst-case) bound because at any given moment each sensor could independently be at the peak of its burst:

$$\alpha_{\text{agg}}(t) = \sum_{k=1}^{n_s} \alpha_s(t) = n_s \cdot \alpha_s(t).$$

Since all 8 sensors are identical:

$$\alpha_{\text{agg}}(t) = 8 \cdot \left[\frac{100(N+1)}{N} \cdot t + 100(N+1) \right].$$

$$\alpha_{\text{agg}}(t) = \frac{800(N+1)}{N} \cdot t + 800(N+1) \quad [\text{bits, with } t \text{ in ms}].$$

- $\frac{800(N+1)}{N}$ — aggregate token rate in kbps; the total long-term rate offered to the link by all 8 sensors. This is exactly r_{total} from Task 1.
- $800(N+1)$ bits — aggregate burst; the maximum number of bits that could arrive simultaneously if all 8 sensors released a packet at the exact same instant.

Consistency check — using $N = 4$ (the theoretical answer). Substituting $N = 4$: aggregate rate = $800 \times 5/4 = 1000$ kbps = s , aggregate burst = $800 \times 5 = 4000$ bits. The aggregate arrival rate *exactly equals* the link speed, which is consistent with $\rho = 1.0$ from Task 1.

For $N = 4$, the aggregate arrival curve and the link's **service curve** $\beta(t) = s \cdot t = 1000t$ can be compared directly.

At $t = 0$ the arrival curve starts $b_{\text{agg}} = 4000$ bits above the service curve — the worst-case burst from all 8 sensors releasing simultaneously. This is the maximum backlog the link must absorb.

After $t = 0$, however, the two curves rise at *exactly the same slope*:

$$r_{\text{agg}} = \left. \frac{800(N+1)}{N} \right|_{N=4} = \frac{800 \times 5}{4} = 1000 \text{ kbps} = s.$$

The net drain rate is therefore:

$$s - r_{\text{agg}} = 1000 - 1000 = 0 \text{ kbps.}$$

The two curves are **parallel** — the gap never closes. Setting $\beta(t^*) = \alpha_{\text{agg}}(t^*)$:

$$1000t^* = 1000t^* + 4000 \implies 0 = 4000 \implies \boxed{t^* = \infty.}$$

The service curve *never catches up* to the arrival curve. Any burst that enters the queue at $N = 4$ stays there permanently. This is the graphical confirmation of what the warnbox in Task 1 stated: a utilisation of exactly 1 means the system has zero capacity to absorb bursts.

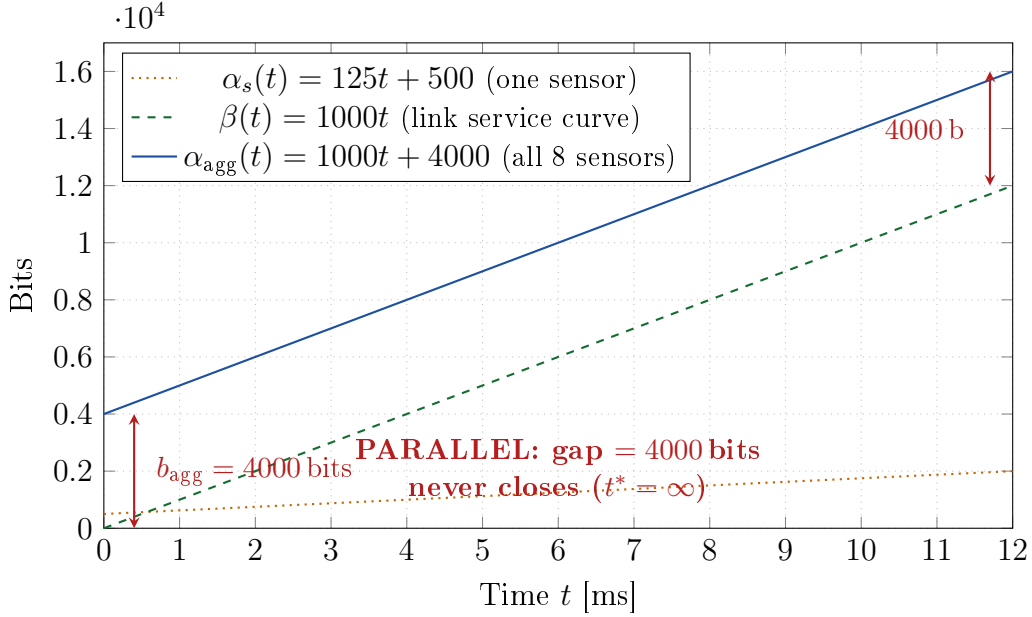


Figure 2: DNC arrival and service curves for $N = 4$ ($\rho = 1.0$). The **blue solid** line ($\alpha_{\text{agg}} = 1000t + 4000$) and the **green dashed** line ($\beta = 1000t$) have *identical slopes* — they are **parallel**. The vertical gap of $b_{\text{agg}} = 4000$ bits (red arrows) is constant for all t : the link can never drain the burst backlog. Any worst-case burst that occurs at $N = 4$ is permanent. The **orange dotted** line is a single sensor's arrival curve.

What the arrival/service curve figure tells us

Reading the DNC curve — key numbers for $N = 4$:

1. **Maximum backlog** $= b_{\text{agg}} = 800(N + 1)|_{N=4} = 4000$ bits

This is the vertical gap at $t = 0$: the worst-case burst when all 8 sensors fire simultaneously (e.g. right after a brief link hiccup lasting less than $P = 4$ ms). The link needs $4000/1000 = 4$ ms of transmission time just to clear this burst — and at $N = 4$ it *never gets the chance to*.

2. **Net drain rate** $= s - r_{\text{agg}} = 1000 - 1000 = 0$ kbps

Both curves have slope 1000 kbps. There is zero spare capacity; the link is fully occupied just keeping up with the steady-state traffic and has nothing left to drain a burst.

3. **Time to clear the burst** $= t^* = b_{\text{agg}}/(s - r_{\text{agg}}) = 4000/0 = \infty$

The gap never closes. If a burst of 4000 bits ever forms in the queue, it stays there forever. The figure makes this visible: the red gap arrows are the *same size* at $t = 0$ and $t = 12$ ms.

4. **Maximum delay per packet** $= b_{\text{agg}}/s = 4000/1000 = 4$ ms

In DNC the worst-case queuing delay equals b/s (the horizontal distance between the two curves). A packet arriving behind a full 4000-bit burst queue must wait 4 ms for it to drain, plus $L/s = 500/1000 = 0.5$ ms for its own serialisation. Adding the packetisation delay $d_{\text{pkt}} = 3$ ms gives a total worst-case delay of 7.5 ms — which is consistent with (and slightly above) the $D(4) = 7.0$ ms from Task 1.

Why this argues for $N = 5$. Switching from $N = 4$ to $N = 5$ changes r_{agg} from 1000 kbps to 960 kbps, giving a net drain rate of 40 kbps instead of zero. The burst is now cleared in $t^* = 4800/40 = 120$ ms instead of never. The cost is 1.8 ms of extra end-to-end delay. That trade-off — 120 ms burst recovery time versus 1.8 ms extra latency — is what makes $N = 5$ the practical recommendation.

Summary

Task 1 — Optimal packet size:

Packet length: $L = 100(N + 1)$ bits. Inter-packet period per sensor: $P = N$ ms.

Bandwidth constraint: $\rho = \frac{0.8(N + 1)}{N} \leq 1 \Rightarrow N \geq 4$.

Worst-case delay: $D(N) = 1.8N - 0.2$ ms — strictly increasing in N .

$\Rightarrow$ **Theoretical answer:** $N = 4$ ($\rho = 1.0$, $D = 7.0$ ms). Under the idealised assumptions of the exercise (perfectly periodic, no jitter) this satisfies the constraint with minimum delay.

$\Rightarrow$ **Practical recommendation:** $N = 5$ ($\rho = 0.96$, $D = 8.8$ ms). Designing to $\rho = 1$ leaves no headroom for jitter, timing imperfections, or any other traffic; $N = 5$ adds only 1.8 ms of delay while providing a meaningful safety margin.

Task 2 — DNC arrival curves (general N):

Single sensor (token bucket $\alpha = r \cdot t + b$):

$$\alpha_s(t) = \frac{100(N + 1)}{N}t + 100(N + 1) \quad [\text{bits}, t \text{ in ms}]$$

Aggregate for all 8 sensors:

$$\alpha_{\text{agg}}(t) = \frac{800(N+1)}{N} t + 800(N+1) \quad [\text{bits}, t \text{ in ms}]$$

For $N = 4$ (theoretical answer): $\alpha_s(t) = 125t + 500$ and $\alpha_{\text{agg}}(t) = 1000t + 4000$.

Curves **parallel** to service curve $\beta = 1000t$ — burst never drains ($t^* = \infty$).

For $N = 5$ (practical recommendation): $\alpha_s(t) = 120t + 600$ and $\alpha_{\text{agg}}(t) = 960t + 4800$.

Net drain rate = 40 kbps, burst clears after $t^* = 120$ ms.

Exercise 2 RTC Toolbox Analysis of the Sensor Network

Note — this exercise is entirely code-based. All results are computed by the RTC Toolbox running in MATLAB. The code file is `ex2_rtc.m` in the same folder. Before running it, ensure the toolbox is installed and call `rtc_init` to load the Java kernel.

Problem Statement

Using the RTC Toolbox, encode the arrival and service curves derived in Exercise 1 and use them to compute delay and backlog bounds programmatically. Specifically:

1. Encode the aggregated arrival constraint for all 8 sensors as an `rtccurve` object.
2. Encode the service curve of the communication link using the toolbox (the lower bounded-delay service curve).
3. Use `rtch` and `rtcplotv` to compute and plot the worst-case **delay** bound.
4. Use `rtcv` and `rtcplotv` to compute and plot the worst-case **backlog** bound.
5. Use an appropriate sweep over N with `rtch` to identify the optimal value of N .

RTC Toolbox Functions Used

Key functions in this exercise:

- `rtccurve([[x0 y0 slope; ...]])` — creates an arbitrary piecewise-linear curve from a matrix of segments. Each row `[x0 y0 s]` defines a segment starting at (x_0, y_0) with slope s . A single row `[[0 b r]]` gives the token bucket $\alpha(t) = r \cdot t + b$.
- `rtcbdl(D, B)` — lower bounded-delay service curve: $\beta(t) = B \cdot (t - D)^+$. With $D = 0$ this simplifies to the pure-rate curve $\beta(t) = B \cdot t$.
- `rtch(alpha, beta)` — maximum *horizontal* distance between the two curves (= worst-case queuing delay bound). Returns `Inf` if the arrival rate exceeds the service rate ($r > s$, unstable system).
- `rtcplotv(alpha, beta)` — draws a **single horizontal line segment** on the current axes marking *where* the maximum horizontal distance occurs (bit level on y -axis, time on x -axis). It does **not** plot delay as a function of time.
- `rtcv(alpha, beta)` — maximum *vertical* distance (= worst-case backlog / buffer bound). Returns `Inf` if $r > s$.
- `rtcplotv(alpha, beta)` — draws a **single vertical line segment** marking *where* the maximum vertical distance occurs (a spike at $t = 0$ for a token-bucket source). It does **not** plot backlog as a function of time.

Why do `rtcploth`/`rtcplotv` look unexpected?

Under the hood, `rtcploth` calls `CurveMath.maxHDistLine` and `rtcplotv` calls `CurveMath.maxVDistLine`. Each returns a single `Vector` object representing *one line segment* that marks the location of the worst-case deviation:

- `rtcplotv` draws a **vertical spike at $t = 0$** from $y = 0$ to $y = b_{\text{agg}}$ (the initial burst). This is correct — the worst-case backlog *does* occur at $t = 0$ when all 8 sensors fire simultaneously. But because it draws only this one marker, you cannot see the subsequent decrease of the backlog over time.
- `rtcploth` draws a **horizontal line at $y = b_{\text{agg}}$ bits** spanning from $t = 0$ to $t = b_{\text{agg}}/s$ (the worst-case delay). The line is clipped to the current axis window, which is typically only 0–1 ms, so it appears very short or flat.

To see how the backlog and delay *evolve over time*, you need to plot $\alpha(t)$ and $\beta(t)$ together on the same axes over a wide time range (Figures 2 and 3 in the code).

Encoding the Curves ($N = 4$)

From Exercise 1 with $N = 4$:

- $L = 100(N + 1) = 500$ bits, $P = N \cdot T = 4$ ms
- $r_{\text{agg}} = 8 \cdot L/P = 1000$ kbps, $b_{\text{agg}} = 8 \cdot L = 4000$ bits

The token-bucket arrival curve $\alpha_{\text{agg}}(t) = 1000t + 4000$ and the pure-rate service curve $\beta(t) = 1000t$ are created as:

```
1 % Token bucket: starts at y = b_agg, rises with slope r_agg
2 % rtccurve([[x0 y0 slope]])
3 alpha_agg = rtccurve([[0 4000 1000]]);
4
5 % Lower bounded-delay service: delay=0, bandwidth=s=1000 kbps
6 % With D=0: beta(t) = 1000*t
7 beta = rtcbd1(0, 1000);
```

Delay and Backlog Bounds

```
1 % Worst-case delay bound (single scalar)
2 delay = rtch(alpha_agg, beta); % N=4: 4.00 ms (= b_agg / s =
   4000/1000)
3
4 % Worst-case backlog bound (single scalar)
5 backlog = rtcv(alpha_agg, beta); % N=4: 4000 bits (= b_agg,
   constant forever)
6
7 % Figure 1: overlay rtcploth / rtcplotv markers ON the actual curve
   plot
8 % so you can see what each marker represents geometrically
9 figure(1);
10 t_ax = 0:0.1:10;
11 plot(t_ax, s*t_ax, 'r--', 'LineWidth', 1.5); % beta
12 hold on;
13 plot(t_ax, r_agg*t_ax + b_agg, 'b-', 'LineWidth', 1.5); % alpha
```

```

14 rtcplov(alpha_agg, beta); % vertical spike at t=0 marking max
    backlog
15 rtcploth(alpha_agg, beta); % horizontal line at y=b_agg marking max
    delay
16 hold off;

```

Interpreting the $N=4$ result. Both curves have slope 1000 kbps so they are *parallel* — the horizontal distance is a constant $b_{\text{agg}}/s = 4$ ms for all t , and the vertical gap is a constant 4000 bits. The delay is *finite*, but the backlog *never drains*: any burst of 4000 bits stays in the queue permanently because there is zero spare capacity ($\rho = 1$).

Time Evolution of Backlog and Delay (Figures 2 & 3)

The backlog at time t is simply $\alpha(t) - \beta(t) = (r_{\text{agg}} - s)t + b_{\text{agg}}$ (clamped to zero once it reaches zero). For the delay at time t , a burst arriving at t into a queue of depth $\max(0, b_{\text{agg}} - (s - r_{\text{agg}})t)$ bits must wait for those bits to be transmitted first:

$$d(t) = \frac{\max(0, b_{\text{agg}} - (s - r_{\text{agg}})t)}{s}.$$

- $d(t)$ — delay experienced by traffic arriving at time t [ms].
- b_{agg} — aggregate burst [bits]; the initial queue depth.
- $s - r_{\text{agg}}$ — net drain rate [kbps]; how fast the queue shrinks.
- s — link speed used to convert bits of queue to waiting time.

These are plotted directly in the code for $N \in \{3, 4, 5\}$:

```

1 t_plot = 0:0.5:160; % 0 to 160 ms
2
3 for Nk = [3, 4, 5]
4     r_k = 8 * (100*Nk + 100) / (Nk * 1); % aggregate rate
5     b_k = 8 * (100*Nk + 100); % aggregate burst
6
7     % Backlog(t): decays once link has spare capacity
8     backlog_t = max((r_k - s)*t_plot + b_k, 0);
9
10    % Delay(t): how long a bit arriving at time t waits
11    delay_t = max(b_k - (s - r_k)*t_plot, 0) / s;
12 end

```

What Figure 2 (backlog vs. time) shows:

- $N=3$ ($r = 1067$ kbps $> s$): backlog grows linearly without bound — the system is unstable.
- $N=4$ ($r = s = 1000$ kbps): backlog stays flat at 4000 bits forever — the parallel-curve case. `rtcv` returns 4000, a finite value, but the queue never clears.
- $N=5$ ($r = 960$ kbps $< s$): backlog starts at 4800 bits at $t = 0$ and *decreases* linearly to 0 at $t^* = 4800/40 = 120$ ms. This is the recovery the warnbox in Exercise 1 was

referring to.

The same pattern holds for Figure 3 (delay vs. time): $N=3$ diverges, $N=4$ stays at 4 ms forever, $N=5$ decreases from 4.8 ms to 0 at $t = 120$ ms.

Finding the Optimal N

The code sweeps $N = 1, \dots, 8$. `rtch` returns `Inf` whenever $r_{\text{agg}} > s$, which flags the system as unstable. The smallest N producing a finite result is the answer.

```
1 for N_val = 1:8
2     L_n = 100*N_val + 100;           % packet length [bits]
3     r_n = 8 * L_n / (N_val * 1);    % aggregate rate [kbps]
4     b_n = 8 * L_n;                  % aggregate burst [bits]
5
6     a_n = rtccurve([[0 b_n r_n]]);   % token bucket for this N
7     d_n = rtch(a_n, beta);           % delay bound (Inf if
                                     % unstable)
8     v_n = rtcv(a_n, beta);           % backlog bound (Inf if
                                     % unstable)
9
10    if isinf(d_n)
11        fprintf('N=%d: UNSTABLE (r=%.0f > s=1000)\n', N_val, r_n);
12    else
13        fprintf('N=%d: delay=%.1f ms, backlog=%.0f bits\n', N_val,
14                d_n, v_n);
15    end
end
```

Expected output:

- $N = 1$: UNSTABLE ($r = 1600 > 1000$)
- $N = 2$: UNSTABLE ($r = 1200 > 1000$)
- $N = 3$: UNSTABLE ($r \approx 1067 > 1000$)
- $N = 4$: delay = 4.0 ms, backlog = 4000 bits (*theoretical minimum, $\rho = 1$*)
- $N = 5$: delay = 4.8 ms, backlog = 4800 bits (*practical recommendation, $\rho = 0.96$*)
- $N \geq 6$: larger delay, smaller ρ — no benefit over $N = 5$

Optimal $N = 4$ from the RTC toolbox perspective (minimum finite delay). Use $N = 5$ in practice to give the link spare capacity.

Exercise 3 CAN Bus RTC Analysis with Priority Scheduling

Problem Statement

Three nodes share a single CAN bus segment. In CAN a *lower* identifier means *higher* priority:

Node	Period	CAN ID	Priority
0	$T_0 = 2 \text{ ms}$	0x000	Highest
2	$T_2 = 3 \text{ ms}$	0x002	Lowest
1	$T_1 = 5 \text{ ms}$	0x001	Middle

All packets are $L = 100$ bits. The code file is `ex3_rtc.m`.

Tasks:

1. Determine the **smallest bitrate** C for the traffic to be executable (all messages meet their deadlines).
2. Build an **RTC model** for the three nodes and the CAN bus. Include a receive buffer at the lowest-priority node drained by a periodic reader with period T_{rx} .
3. Determine the **maximum point-to-point delay** for Node 2 and the **minimum receive-buffer size** for arbitrary T_{rx} and C .

Part 1 — Minimum Bitrate

The total offered load must not exceed the link capacity. Summing over all three nodes:

$$\rho = \frac{L}{T_0 \cdot C} + \frac{L}{T_1 \cdot C} + \frac{L}{T_2 \cdot C} = \frac{L}{C} \left(\frac{1}{T_0} + \frac{1}{T_1} + \frac{1}{T_2} \right) \leq 1.$$

- ρ — link utilisation; must be ≤ 1 for schedulability.
- C — CAN bus bitrate in kbps.
- $L = 100$ bits — packet length.
- T_0, T_1, T_2 — periods in ms.

Solving for the minimum bitrate:

$$C_{\min} = L \left(\frac{1}{T_0} + \frac{1}{T_1} + \frac{1}{T_2} \right) = 100 \left(\frac{1}{2} + \frac{1}{5} + \frac{1}{3} \right) = 100 \cdot \frac{31}{30} \approx 103.33 \text{ kbps}.$$

In the code:

```
1 C_min = L * (1/T0 + 1/T1 + 1/T2);    % = 103.33 kbps
```

At $C = C_{\min}$ the utilisation is exactly $\rho = 1$ (same issue as $N = 4$ in Exercise 2). Choose C strictly greater than 103.33 kbps. The parametric analysis in Part 3 sweeps C from 110 to 1000 kbps.

Part 2 — RTC Model and Priority Scheduling

Additional RTC functions used in Exercise 3:

- `rtcpjdu(P)` / `rtcpjdl(P)` — upper/lower *event arrival curves* for a periodic stream with integer period P and zero jitter. These count *packets* (events), not bits.
- `rtcbdu(D, B)` — upper bounded-delay service curve (matches `rtcbd1` but gives an upper bound).
- `rtcgpc(AU, AL, BU, BL, ED)` — **Greedy Processing Component**: processes the event stream $[AU, AL]$ on the resource $[BU, BL]$ with execution demand ED bits per event. Returns:
 - `[AU_out, AL_out]` — output event arrival curves.
 - `[BU_res, BL_res]` — residual service curves (what is left for lower-priority flows).
 - `del` — worst-case delay for this flow [ms].
 - `buf` — worst-case buffer [events].
- `rtcpsl(S, P, B)` — lower periodic service curve: allocates a share S per period P on a resource of bandwidth B . Used here for the periodic receiver at Node 2.

Arrival curves. Each node sends one L -bit packet every T_i ms. The upper PJD curve (Period-Jitter-Distance) with $J = 0$ and $D = 0$ bounds the number of packets in any window of length t by $\lceil t/T_i \rceil$:

```
1 alpha0_u = rtcpjdu(T0); alpha0_l = rtcpjdl(T0); % Node 0 (T0=2ms)
2 alpha1_u = rtcpjdu(T1); alpha1_l = rtcpjdl(T1); % Node 1 (T1=5ms)
3 alpha2_u = rtcpjdu(T2); alpha2_l = rtcpjdl(T2); % Node 2 (T2=3ms)
```

Service curve. The CAN bus provides C kbps continuously:

```
1 beta_u = rtcbdu(0, C); % upper: beta(t) = C*t
2 beta_l = rtcbd1(0, C); % lower: beta(t) = C*t (D=0 so both equal)
```

Priority scheduling as a GPC chain. Strict-priority scheduling is modelled by chaining three GPCs. Each GPC consumes L bits of service per packet ($ED = L$). The *residual* service left by higher-priority nodes flows as input to the next GPC:

```
1 % Node 0 (highest priority) -- gets the full CAN bus service
2 [a0u_out, a0l_out, b0u_res, b0l_res, del0, buf0] = ...
3   rtcgpc(alpha0_u, alpha0_l, beta_u, beta_l, L);
4
5 % Node 1 (second priority) -- gets what is left after Node 0
6 [a1u_out, a1l_out, b1u_res, b1l_res, del1, buf1] = ...
7   rtcgpc(alpha1_u, alpha1_l, b0u_res, b0l_res, L);
8
9 % Node 2 (lowest priority) -- gets the residual after Nodes 0 and 1
```



```

10 [a2u_out, a2l_out, b2u_res, b2l_res, del2, buf2] = ...
11   rtcgpc(alpha2_u, alpha2_l, b1u_res, b1l_res, L);

```

Why the GPC chain captures strict priority. The `rtcgpc` function models a work-conserving processor. When Node 0 runs at the highest priority its packets consume service from the full curve $[\beta_U, \beta_L]$. The residual $[BU_{\text{res}}, BL_{\text{res}}]$ is the service *that would remain* if Node 0 were never idle — i.e. the worst-case interference seen by Node 1. Chaining three GPCs therefore gives the tightest achievable RTC bound on the lower-priority nodes' delay and buffer without simulating every possible scheduling scenario.

Receive buffer. The departures of Node 2's packets (`a2u_out`) accumulate in a buffer drained by a periodic reader with period T_{rx} . The reader is modelled by a lower periodic service curve `rtcpsl`(S , P , B) with share $S = 1$ (one read per period), period $P = T_{\text{rx}}$, and unit bandwidth $B = 1$:

```

1 beta_rx_l = rtcpsl(1, T_rx, 1);      % lower service: 1 pkt per T_rx ms
2 buf_min   = rtcv(a2u_out, beta_rx_l); % max backlog = min buffer size

```

Part 3 — Results and Parametric Analysis

Expected results for $C = 200$ kbps, $T_{\text{rx}} = 10$ ms:

- Node 0 (highest priority, $T_0 = 2$ ms): delay close to one transmission time $L/C = 0.5$ ms.
- Node 1 (second priority, $T_1 = 5$ ms): delay includes worst-case interference from Node 0.
- Node 2 (lowest priority, $T_2 = 3$ ms): **longest delay** — suffers interference from both higher-priority nodes.
- Minimum receive buffer: increases with T_{rx} (slower reader $\rightarrow$ more packets accumulate between reads).

The code sweeps $T_{\text{rx}} \in \{2, 3, 4, 5, 6, 8, 10, 15, 20\}$ ms and $C \in \{110, 120, 150, 200, 300, 500, 1000\}$ kbps to produce a complete picture of how delay and buffer depend on the two free parameters. Key trends:

- **Increasing C** reduces all delays (more service available per unit time), but returns diminish quickly above ~ 300 kbps.
- **Increasing T_{rx}** directly increases the minimum buffer because more packets can arrive between consecutive reads.
- **If $T_{\text{rx}} > T_2$** the reader is slower than the source: the buffer grows without bound (`rtcv` returns `Inf`). The receiver period must satisfy $T_{\text{rx}} \leq T_2 = 3$ ms for lossless operation (or a larger buffer must be provisioned).